SF2957 - project 2

Group 9: Tommaso Brambilla, Ramon Padilla, Matilde Reina, Valentina Tomassini

December 2025

Setup of the problem

In this assignment, we want to use reinforcement learning to train an agent to play blackjack. The setup of the problem is the following:

- **Episode:** an episode is one complete hand of Blackjack (dealing initial cards → player's decisions → dealer's play → final win/loss)
- **Sum-based State space S_2:** A state is a 3-tuple: (S_a, u, S_d), where:
  - S_a: cards' sum of the agent
  - S_d: cards' sum of the dealer
  - u: player is holding a usable ace ($u = 1$) or not ($u = 0$)
- **Extended State space S_1:** A state is a 2-tuple: (S_a, S_d), where:
  - S_a is an array of 10 elements, where the ith element S_{a_i} is the number of cards of value i
  - S_d: card's sum of the dealer
- **Action space:** Hit (1) or Stick (0)

Given the state space $\mathcal{S}$, the action space $\mathcal{A} = \{0, 1\}$ (stick/hit) and corresponding rewards $r \in \{-1, 0, +1, +1.5\}$, we want to find the optimal policy π^*.

1 On-policy Monte Carlo Control

Implementation available in Appendix.

1.1 Action-value Estimation

An *on-policy* method learns the value of the policy π that is currently being used to generate behavior. For a policy π, the action-value function is defined as:

$$q^\pi(s, a) = E_\pi \left[G_t \mid S_t = s, A_t = a \right],$$

where $G_t = \sum_{k=t}^{T-1} \gamma^k R_{t+k+1}$ is the *return* from time t (with $\gamma = 1$ in our case).

In Monte Carlo methods, $q^\pi(s, a)$ is estimated by the *sample average* of returns following all visits to (s, a):

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s,a)} G^{(i)}(s, a),$$

where $N(s, a)$ is the number of times (s, a) has been visited, and $G^{(i)}(s, a)$ is the return following the i-th visit to (s, a).

1.2 Incremental Update Formula

After the $N(s,a)$-th visit yielding return G, the update is:

$$Q_{N+1}(s,a) = Q_N(s,a) + \frac{1}{N(s,a)+1}\big(G - Q_N(s,a)\big) \tag{1}$$

$$= Q_N(s,a) + \frac{G - Q_N(s,a)}{N(s,a)+1}. \tag{2}$$

In our blackjack implementation ($\gamma = 1$, terminal rewards only), for any (s,a) visited in episode i, the return $G^{(i)}$ equals the *episode reward* G_i (final payout).

1.3 ϵ-soft Exploration

To ensure all (s,a) pairs are eventually visited (eliminating the exploring-starts assumption), we use an *ϵ-greedy* policy:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg\max_{a'} Q(s,a'), \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases}$$

In code: $\epsilon = 0.05$, $|\mathcal{A}| = 2$.

With decaying exploration: $\epsilon_t = \epsilon/(1 + N(s)_{\text{total}})$.

2 Q-Learning

We again consider blackjack as a finite episodic MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with $\gamma = 1$. The action set is $\mathcal{A} = \{0, 1\}$ corresponding to *stick* and *hit*. Implementation available in Appendix.

2.1 Action-value function and optimality

For any policy π, the action-value function is

$$q^\pi(s,a) = E_\pi\big[G_t \,\big|\, S_t = s,\ A_t = a\big],$$

where

$$G_t = \sum_{k=0}^{\tau-t-1} \gamma^k R_{t+k+1}$$

is the return from time t until the terminal time τ.

The optimal action-value function is

$$q^*(s,a) = \sup_\pi q^\pi(s,a).$$

It satisfies the Bellman optimality equation

$$q^*(s,a) = E\Big[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \,\Big|\, S_t = s, A_t = a\Big].$$

Any greedy policy with respect to q^*,

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} q^*(s,a),$$

is optimal.

2.2 Q-learning update rule

Q-learning constructs a sequence of estimates $\{q_t\}$ that aims to converge to q^* by *bootstrapping* from sampled transitions. Given a transition $(S_t, A_t, R_{t+1}, S_{t+1})$, the basic Q-learning update is

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(S_t, A_t)\Big(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t)\Big).$$

Here:

- $R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a')$ is the *TD target*, an estimate of the optimal action value,
- $\alpha_t(S_t, A_t)$ is the *step-size* (learning rate) for the current stateaction pair.

In blackjack, with $\gamma = 1$ and only terminal rewards, this becomes:

- For non-terminal S_{t+1}: target $= \max_{a'} q_t(S_{t+1}, a')$.
- For terminal S_{t+1}: target $= R_{t+1}$ (no future term).

2.3 Step-size schedule

The step-size values follow the general sequence $\{\alpha_n\}$ with $\alpha_n = cn^{-\omega}$. In our code, we use

$$\alpha_n = \frac{1}{n^\omega}, \qquad 0 \le \omega < 1,$$

where $n = N_t(s, a)$ is the number of times the current stateaction pair (s, a) has been visited before.

So the effective step-size in the update

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big(\text{target} - Q(s, a)\big)$$

decays as that particular (s, a) pair is sampled more often, stabilizing the estimate.

2.3.1 Behaviour policy: ε-greedy

Q-learning is *off-policy*: it learns q^* while following an exploratory behaviour policy.

At time t, given current estimate Q, we form the ε-greedy policy

$$\pi_t(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|}, & a \in \arg\max_{a'} Q(s, a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|}, & \text{otherwise.} \end{cases}$$

We then:

1. Draw A_t from $\pi_t(\cdot \mid S_t)$,
2. Observe R_{t+1}, S_{t+1},
3. Apply the Q-learning update above.

3 Approach comparison

First we analyze what results to expect on a theoretical level. In particular, we are asked to compare the performances of on-policy MC learning algorithm with state space $S1$, Q-learning with state space $S1$ and Q-learning with state space $S2$.

On-policy MC vs. Q-learning on a single hand

Consider the sum-based state representation S_2, with states $s = (S_a, S_d, u)$. Suppose a particular episode (hand) evolves as follows:

$$s_0 = (12, 6, 0) \xrightarrow{a_0=\text{hit}} s_1 = (17, 6, 0) \xrightarrow{a_1=\text{stay}} s_2 = \dagger,$$

where $\dagger$ is the terminal state. Assume the intermediate reward is zero (no payout during the hand) and the final reward is $+1$ (the player wins when standing on 17 against dealer 6):

$$R_1 = 0, \qquad R_2 = +1.$$

Let $\gamma = 1$.

On-policy Monte Carlo update. In on-policy MC, there are no updates during the hand; we wait until the episode terminates. The return for this episode is

$$G = R_2 = +1.$$

If this is the $N(s, a)$-th visit to each of the stateaction pairs (s_0, hit) and (s_1, stay), the MC updates are

$$Q_{\text{MC}}(s_0, \text{hit}) \leftarrow Q_{\text{MC}}(s_0, \text{hit}) + \frac{1}{N(s_0, \text{hit})}\big(G - Q_{\text{MC}}(s_0, \text{hit})\big),$$

$$Q_{\text{MC}}(s_1, \text{stay}) \leftarrow Q_{\text{MC}}(s_1, \text{stay}) + \frac{1}{N(s_1, \text{stay})}\big(G - Q_{\text{MC}}(s_1, \text{stay})\big).$$

Both decisions in the hand are "inherit" the full win $+1$

Q-learning update. In Q-learning, we update after each transition. Suppose that before this episode we have some current estimates $Q(s, a)$. At the first step, $(s_0, a_0) = ((12, 6, 0), \text{hit})$ leads to $s_1 = (17, 6, 0)$ with reward $R_1 = 0$. The Q-learning update is

$$Q(s_0, \text{hit}) \leftarrow Q(s_0, \text{hit}) + \alpha\Big(0 + \gamma \max_{a'} Q(s_1, a') - Q(s_0, \text{hit})\Big).$$

Here the target $\max_{a'} Q(s_1, a')$ is the current estimate of the value of the best action at $(17, 6, 0)$, obtained from the existing Q-table (learned from previous episodes). Thus the value of hitting on 12 is moved towards what we currently believe we can achieve from the subsequent state s_1.

At the second step, from $s_1 = (17, 6, 0)$ taking $a_1 = \text{stay}$ yields reward $R_2 = +1$ and transitions to the terminal state. Here, with $\gamma = 1$, the update is

$$Q(s_1, \text{stay}) \leftarrow Q(s_1, \text{stay}) + \alpha\big(1 + 0 - Q(s_1, \text{stay})\big) = Q(s_1, \text{stay}) + \alpha\big(1 - Q(s_1, \text{stay})\big),$$

since there is no future term after termination.

In summary, MC assigns the full observed return $G = +1$ to all visited stateaction pairs only after the episode ends, whereas Q-learning propagates value step-by-step: the terminal reward first updates $Q(17, 6, 0, \text{stay})$, and this updated estimate then influences future updates of $Q(12, 6, 0, \text{hit})$ through the bootstrapped term $\max_{a'} Q(17, 6, 0, a')$. This temporal-difference mechanism underlies the different learning dynamics we observe in the experiments.

State spaces S_1 vs. S_2 (Q-learning) The Bellman optimality equation is the same in both cases, but the state space changes. On S_1, $Q(s, a)$ can in principle encode subtle dependence on hand composition (e.g. distinguishing $16 = 10 + 6$ from $16 = 8 + 8$), whereas on S_2 many different hands are aggregated into the same state and Q-learning learns an average rule over those hands. Since blackjack decisions are essentially determined by (S_a, S_d, u), we expect Q-learning on S_2 to converge faster and more stably (fewer states, more visits per state, lower variance), while Q-learning on S_1 is expected to be slower and noisier but could, in principle, achieve slightly more refined strategies given very large sample sizes.

4 Results

4.1 Model comparison

In this initial experiment, the performance of the three models is compared across deck sizes, we use constant exploration.

Table 1: Comparison of Average Rewards across different deck counts and learning algorithms (with 10^5 generated episodes). Constant exploration.

Decks	MC Extended	Q Extended	Q Sum
1	-0.23373	-0.05333	-0.03645
2	-0.24489	-0.07499	-0.06468
6	-0.25909	-0.08476	-0.06593
8	-0.26167	-0.08928	-0.06050
Inf	-0.27124	-0.09466	-0.06691

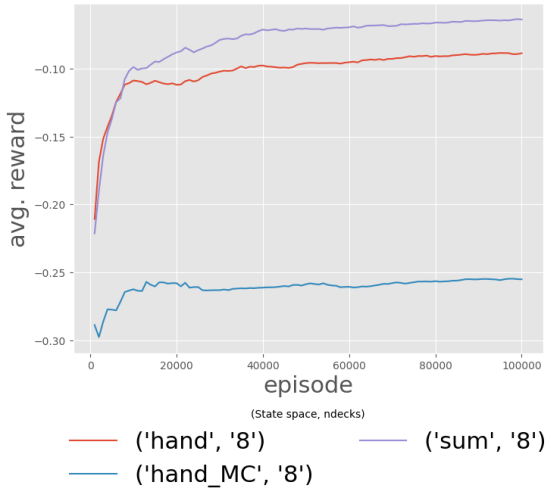


Figure 1: Comparison of the average reward for the three different methods. Constant exploration.

Table 1 compares the performance of the three learning configurations across all deck sizes. Q-learning on the reduced sum state space $\boldsymbol{S}_2$ achieves the highest average reward in every setting. This result is coherent with our expectation that given this state space contains far fewer states than the extended representation $\boldsymbol{S}_1$, it results in many more visits per state-action pair, and therefore faster convergence. Monte Carlo performs worse because its updates occur only at the end of each episode and rely on a noisy return, resulting in high variance. These results can also be observe in the plot Figure 1.

4.2 Step-size $\alpha_n = cn^{-\omega}$.

For a given stateaction pair (s, a) visited n times, we use

$$\alpha_n = \frac{c}{n^{\omega}}, \qquad 0 \leq \omega < 1.$$

Each Q-update is a weighted average between the old estimate and a new TD target; α_n controls how much weight is placed on new information. Smaller ω (slow decay) yields faster adaptation but more oscillation and

noise; larger ω (fast decay) yields slower adaptation but more stable estimates. Because blackjack rewards are noisy (win/loss/push), too small ω can make learning unstable, while too large ω makes it difficult to correct early errors. An intermediate ω should balance adaptation and stability.

Table 2: Average Reward for 8 decks with varying learning rate decay ω.

Omega	**Q-Extended**	**Q-Sum**	**MC (Ref)**
0.50	-0.09599	-0.06678	-0.26648
0.65	-0.08805	-0.06748	-0.26648
0.77	-0.08509	-0.05893	-0.26648
0.95	-0.08503	-0.06035	-0.26648
1.00	-0.07323	-0.07624	-0.26648

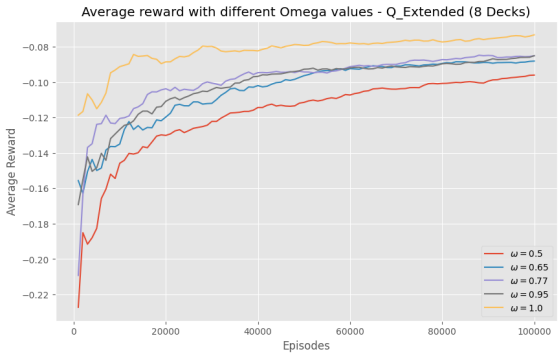


Figure 2: Comparison of the average reward for Q learning method on the extended State space S_1 with different values of ω

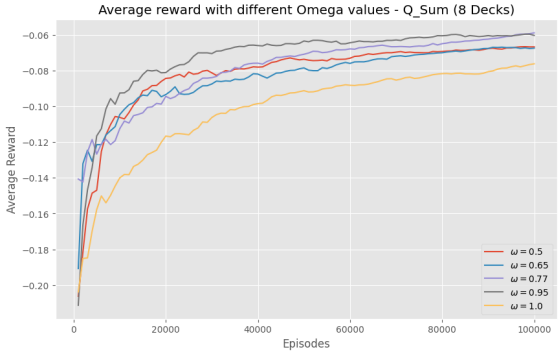


Figure 3: Comparison of the average reward for Q learning method on the sum-based state space S_2 with different values of ω

Table 2 shows the average rewards when we have 8 decks and change the value of ω, while the plots show the trajectory over the episodes for the different values of ω. Results vary significantly across experiments,for example for other deck sizes in section 5. There is indication, however, that as in our assumptions an intermediate $w \in [0.65, 0.77]$ yields the best convergence properties.

4.3 Constant vs. decaying exploration ε.

The exploration rate ε is proportional to the probability of deviating from the greedy action. With constant ε (e.g. $\varepsilon = 0.05$), the behaviour policy remains ε-soft: the agent keeps exploring forever, which reduces the

risk of getting stuck but slightly lowers the asymptotic average return (since suboptimal random actions are always taken with probability ε). With a decaying ε_t (e.g. $\varepsilon_t = \varepsilon_0/(1+N(s))$), the agent explores more in the early phase and gradually becomes more greedy as Q stabilizes. In blackjack this means that, asymptotically, the policy approaches a near-deterministic basic strategy, whereas with constant ε it continues to occasionally hit on strong hands or stand on weak hands purely for exploration.

Table 3: Average reward with decaying ϵ

Decks	MC Extended	Q Extended	Q Sum
1	-0.28428	-0.06266	-0.05296
2	-0.26414	-0.09196	-0.07873
6	-0.27858	-0.10038	-0.06798
8	-0.27695	-0.09499	-0.06661
∞	-0.28669	-0.10351	-0.06592

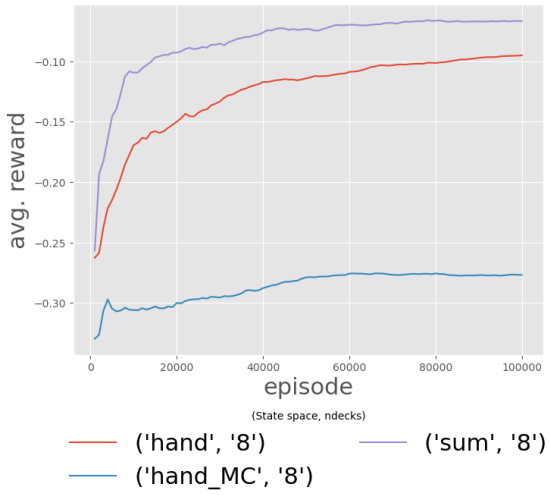


Figure 4: Comparison of the average reward for the three different methods with a decaying ϵ

If we compare Table 1 with Table 3 or Figure 1 with Figure 4, we see that, with the decaying ϵ we obtain better results, as expected. Intuitively, the model explores less from a given state as it visits it, and the estimate improves.

5 Table of actions

In the final part of the project, we used the Q-learning method on S_2 with a decaying ϵ, as we saw in the previous sections that this method was the one that gave us the best results, to find a close to optimal strategy for the game. Below, we attached the table of actions we obtained, both for a usable and non-usable ace. We don't show a strategy when $S_a < 12$, as in that case we always hit and we did not want to make the table redundant.

Table 4: Optimal Policy for Hard Totals (No Usable Ace). H = Hit, S = Stick.

Sum	**A**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
21	S	S	S	S	S	S	S	S	S	S
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	S
16	H	S	S	S	S	S	H	H	H	H
15	H	S	S	S	S	S	H	H	H	H
14	H	S	S	S	S	S	H	H	H	H
13	H	S	S	S	S	S	H	H	H	H
12	H	S	H	S	S	S	H	H	H	H

Table 5: Optimal Policy for Soft Totals (Usable Ace). H = Hit, S = Stick.

Sum	**A**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
21	S	S	S	S	S	S	S	S	S	S
20	S	S	S	S	H	H	H	S	S	S
19	H	S	S	H	H	H	S	S	S	H
18	H	H	H	H	S	H	H	S	H	H
17	S	H	H	H	H	H	H	H	H	H
16	S	H	H	H	H	H	H	H	H	H
15	H	H	H	H	H	H	H	H	H	H
14	H	H	H	H	H	H	H	H	H	H
13	H	H	H	H	H	H	H	H	H	H
12	H	H	H	H	H	H	H	H	H	H

The policies learned by $Q-learning$ on $\boldsymbol{S}_2$ resemble the classical basic strategy used in Blackjack. Hard totals recommend staying from 17 upwards and hitting below 16 (until 12) unless the dealer shows a weak card. Soft totals are more aggressive (as expected) favoring hits on soft 17 or lower. We also observe much more noise in the Soft totals, where the policy has not converged as well.

Appendix

5.1 Code

The implementation in python of the two algorithms.

- **Q-table**: Q is a `defaultdict` with

$$Q[s] \in R^2, \quad Q[s][a] \approx q_t(s, a).$$

- **Visit counts**: `state_action_count[state][action] = n` implements $N_t(s, a)$.

On-policy Monte Carlo learning

```
# Update avg_reward
    avg_reward += (episode_reward - avg_reward) / (episode)

# Update the Q-function for each visited state/action pair.
for state in episode_state_action_count:
    for action in episode_state_action_count[state]:
        G = episode_reward
        N = state_action_count[state][action]
        if N > 0:
            Q[state][action] += (G - Q[state][action])/ N
```

Q-learning

```
# Compute the learning rate
n = state_action_count[state][action]
alpha = 1/(n**omega) #learning rate

if done:
    max_q_Stnext = 0.0 #if the game is done
else:
    max_q_Stnext = np.max(Q[state2])

#Update the Q-function
Q[state][action] += alpha * ( action_reward + gamma * max_q_Stnext - Q[state][action])
```

5.2 Additional Results

Table 6: Average Reward for 1 decks with varying learning rate decay ω.

Omega	Q-Extended	Q-Sum	MC (Ref)
0.50	-0.05781	-0.04275	-0.23180
0.65	-0.05542	-0.04171	-0.23180
0.77	-0.05415	-0.04797	-0.23180
0.95	-0.05852	-0.04070	-0.23180
1.00	-0.05149	-0.04339	-0.23180

Table 7: Average Reward for 2 decks with varying learning rate decay ω.

Omega	**Q-Extended**	**Q-Sum**	**MC (Ref)**
0.50	-0.07636	-0.05622	-0.24670
0.65	-0.07408	-0.05925	-0.24670
0.77	-0.07516	-0.04924	-0.24670
0.95	-0.07298	-0.04974	-0.24670
1.00	-0.07578	-0.05757	-0.24670

Table 8: Average Reward for 6 decks with varying learning rate decay ω.

Omega	**Q-Extended**	**Q-Sum**	**MC (Ref)**
0.50	-0.09177	-0.07033	-0.25766
0.65	-0.08783	-0.05981	-0.25766
0.77	-0.08484	-0.06065	-0.25766
0.95	-0.08313	-0.06263	-0.25766
1.00	-0.08702	-0.05894	-0.25766

Table 9: Average Reward for infinite decks with varying learning rate decay ω.

Omega	**Q-Extended**	**Q-Sum**	**MC (Ref)**
0.50	-0.09438	-0.07017	-0.28337
0.65	-0.09154	-0.06437	-0.28337
0.77	-0.08733	-0.06245	-0.28337
0.95	-0.09574	-0.06847	-0.28337
1.00	-0.08465	-0.07515	-0.28337